How We Trained the Data Leakage Model

A simple guide to the whole idea · Detecting Data Leakage in Cloud Storage

1. The idea in one paragraph

Companies store confidential files in the cloud. Sometimes someone takes data out without permission: a dishonest employee, an attacker with a stolen password, or a careless mistake. We want a computer to **look at each action and say whether it looks like a leak**. To do that, we show it thousands of past actions that are already labelled "normal" or "leak", and it learns the patterns. This is called **training**. Then we check it on actions it has never seen. This is called **testing**.

> **Think of a student.** Training is studying with the answer key. Testing is the final exam with new questions. If you grade a student on questions they already practised, the score means nothing. That is why we keep part of the data hidden until the very end.

2. The data

We used the same Kaggle dataset as the reference paper (Harahap & Hasibuan, 2025): **49,500 cloud-storage events**. Each event is one action by one user, with a label: **normal (68.9%)** or **leak (31.1%)**. We checked that the file matches the paper's own tables row for row.

What the model looks at	Examples
Who the user is	Role: staff, manager, senior manager, intern, lead technician
How they logged in	Password, PIN, multi-factor authentication (yes/no)
What they did	Read, write, move, delete; modified data; accessed or transferred confidential data
How sensitive the data is	Low, medium, high
Added by us: where and when	Internal or external destination; hour of the day

The paper used the first nine attributes. We added the last two because they were in the dataset and they matter: leaks happen about 40% of the time at night against 16% in working hours, and more often when data goes to an external destination.

3. The training process, step by step

1 — Get the data
49,500 labelled events from Kaggle, checked against the paper's tables.

2 — Prepare
Extract the hour from the date; drop ID columns; keep 11 attributes.

3 — Split once
80% for training, 20% locked away as the final exam.

4 — Clean inside
Fill missing values and convert text to numbers, learned from training data only.

5 — Tune
Try 80 tree settings with 5-fold cross-validation; keep the best.

6 — Compare
Check other attributes, models and balancing methods.

7 — Set the alarm
Choose the risk score above which an event raises an alarm.

8 — Final test
Score the locked 20% once, then explain the rules.

All of this runs in one Google Colab notebook, using Python and the scikit-learn library.

4. The key ideas, explained simply

Decision tree. The model asks yes/no questions, like a flowchart: "Was confidential data transferred? Was it sent outside? Was it at night?" At the end it gives a risk score. We chose it because a security analyst can read *why* it raised an alarm.

Train/test split. 80% of events (39,600) are for learning. 20% (9,900) are locked away and used only once, at the end. No decision is ever made using them.

Cross-validation. To compare settings fairly without touching the test set, we split the training data into 5 parts: train on 4, check on the 5th, and repeat 5 times. The average is the score.

Tuning. A tree has settings, such as how many questions in a row it may ask. The computer tried 80 combinations and kept the one with the best cross-validation score.

Class weights. Leaks are the minority (31%), so a lazy model just says "normal". We made each missed leak count about 2.2 times more during training, so the model pays attention to leaks.

Alarm threshold. The tree gives each event a risk score from 0 to 1. We chose the cut-off (0.416) that catches at least 80% of leaks with as few false alarms as possible, using training data only.

No cheating (no data leakage in the method). Filling missing values and converting text to numbers are learned from training data only. Otherwise information from the exam would sneak into studying and inflate the score.

The ceiling. 91% of events look identical to events with the opposite label. No model can tell those apart, so with the paper's nine attributes nothing can pass 74.3% accuracy. Better data matters more than a fancier model.

5. How we measure success

Imagine 100 events: 69 normal and 31 leaks.

Measure	Question it answers
Accuracy	How many answers were right overall? *Misleading here: always saying "normal" already scores 69% and catches no leaks.*
Recall	Of all real leaks, how many did we catch?
Precision	Of all alarms, how many were real leaks?
F1	One score that is high only when recall and precision are both good.
PR-AUC	How well the model ranks risky events above normal ones, whatever the threshold.

6. What we found

Along the way (cross-validation on training data): adding the two new attributes improved the model far more (PR-AUC +0.12) than switching to stronger algorithms such as Random Forest or XGBoost (+0.01). Giving leaks extra weight worked better than creating synthetic leak events (SMOTEN).

Final exam (the 9,900 locked events, scored once):

	Accuracy	Leaks caught (recall)	Precision	F1
Always say "normal"	68.9%	0%	–	0.00
Paper's model (re-checked)	69.5%	10%	55.0%	0.17
Our final model	**69.6%**	**83%**	**50.7%**	**0.63**

Both models have about the same accuracy, but ours catches **about 8 times more leaks**. The price: about half of all events raise an alarm, and about one alarm in two is a real leak. The strongest warning sign the model learned is **a confidential file sent to an external destination**, especially at night.

> **The whole idea in one sentence:** we taught a decision tree to recognise leaks from 39,600 labelled events, made every choice using only that training data, and proved on 9,900 hidden events that it catches far more leaks than the paper's model.

7. Limits to keep in mind

The attributes are simple and many events look alike, so precision stays around 50% at high recall. The dataset's labels are not documented and may be synthetic, so results may not carry over to real cloud logs. A real deployment would need richer behaviour information, such as each user's usual habits.